

Workflow Runbook

Table of contents

Introduction	1
Data and Tools	2
Common Conventions	3
Structure Identification	3
Stage 1: <code>analyze_snapshot.py</code> (build and extract topology)	3
Stage 2: <code>ndtopo_stats.py</code> (overlap/mass attribution)	5
Stage 3: <code>batch_clip.py</code> (slab clip + 2D/3D views)	6
Stage 4: <code>batch_crop_and_clip.py</code> (batch crops + slabs)	8
Stage 5: <code>aggregate_topology_points.py</code> (cross-crop aggregation)	9
Visualization Scripts	10
<code>spin_render.py</code> (rotating 3D render)	10
<code>redshift_evolution.py</code> (redshift evolution movie)	11
<code>compare_simulations.py</code> (side-by-side comparison)	11
Data Analysis Outputs	12
Troubleshooting / Gotchas	12
Technologies Used	12
Materials / Sources	13

Introduction

This project extracts, visualizes, and quantifies cosmic-web topology (walls, filaments, and clusters) from large N-body snapshots using DisPerSE-based workflows. Input snapshots can be provided either as a single HDF5 file or as a directory of split files (e.g., `snapdir_000`), which the reader traverses via header metadata. The pipeline combines command-line scripts to:

- Stream/crop particle data, tessellate density, and extract Morse–Smale structures.
- Clip slabs and flatten outputs for 2D overlays and PNGs.
- Compute overlap/unique/unassigned topology statistics using authoritative Delaunay scalars.
- Batch these steps over many crop tiles and slab positions for large volumes.
- Aggregate statistics and produce histograms, composition plots, and movies across the full dataset.

Pipeline stages:

1. `analyze_snapshot.py` – build NDfield, run Delaunay/MSE, export VTU/VTP.
2. `ndtopo_stats.py` – compute overlap/unique/unassigned counts and scalar sums/means.
3. `batch_clip.py` – clip a slab, flatten, resample/average density, render PNGs.
4. `batch_crop_and_clip.py` – batch driver to tile a large snapshot, run stages 1–3, collect outputs per crop/slab.
5. `aggregate_topology_points.py` – combine per-point CSVs across all crops into global stats, histograms, and composition/violin/box plots.

Visualization scripts (standalone, not part of the batch pipeline):

- `spin_render.py` – rotating 3D camera animation of any VTK dataset → PNG sequence / MP4.
 - `redshift_evolution.py` – smooth interpolated movie across redshift snapshots → MP4.
 - `compare_simulations.py` – side-by-side boxplots comparing two simulations’ topology stats.
-

Data and Tools

- **Data:** Quijote snapshot (e.g., `data/snap_010.hdf5` or a multi-file folder such as `data/snapdir_000`), 1 Gpc/h box, PartType1 dark matter coordinates.
 - **Core tools:**
 - DisPerSE binaries: `del aunay_3D`, `mse`, `netconv`, `skelconv` (on `$PATH` or pointed to via `--disperse-bin-dir`).
 - ParaView `pviewpython` (required for `batch_clip.py`, `spin_render.py`, and `redshift_evolution.py`).
 - Python scripts in `scripts/`: `analyze_snapshot.py`, `batch_clip.py`, `batch_crop_and_clip.py`, `ndtopo_stats.py`, `aggregate_topology_points.py`, `spin_render.py`, `redshift_evolution.py`, `compare_simulations.py`.
 - **Dependencies:** `vtkmodules` (VTK I/O), `hdf5plugin` (Blosc filter for Quijote), `numpy`, `h5py`, `polars` (fast aggregation), `scipy` (proximity alignment in `redshift_evolution.py`).
 - **Units:** Snapshots in kpc/h by default; outputs typically in mpc/h (`--output-unit`). Cropped runs rebase coordinates to the crop origin for periodic tessellation, then shift VTK outputs back to global coordinates.
-

Common Conventions

- Filenames from `analyze_snapshot.py` use simplified stems:
 - Delaunay: `<prefix>_delaunay_S###.vtu`
 - Walls: `<prefix>_manifolds_<TAG>_S###.vtu`
 - Filaments: `<prefix>_arcs_<ARC>_S###.vtp` (a matching `.vtu` is also written)
 - Cluster critical points: `<prefix>_cluster_critpoints_<TAG>_S000.vtp` and `.vtu`
 - Cropped runs: NDfield coordinates are rebased to the crop origin; exported VTU/VTP are shifted back to global.
 - Use `--delaunay-btype periodic` rather than `--periodic` for crops; `--periodic` can introduce artifacts.
-

Structure Identification

- **Walls:** 2D manifolds from the Morse–Smale complex of the DTFE density field. These sheet-like separatrices partition flow basins and naturally trace cosmic walls.
 - **Filaments:** 1D arcs (skeleton) connecting saddle points to extrema. They trace ridges of the density field and are robust under persistence pruning.
 - **Filament manifolds:** Optional 2D manifolds that bound the filaments. Useful for overlap/composition analysis; exported separately via `--dump-filament-manifolds`.
 - **Clusters:** Density maxima (critical points, `type=3`). DisPerSE’s 3-manifold exports are volumetric and often empty after smoothing, so the pipeline exports maxima critical points via `--dump-clusters` and maps them to Delaunay IDs via nearest-neighbor assignment. Cluster `log_field_value` is computed as `ln(field_value)` to match other structures.
 - **Scalars:** All overlap/summary stats use Delaunay scalars, which avoids interpolation artifacts from manifold/skeleton geometry.
-

Stage 1: `analyze_snapshot.py` (build and extract topology)

Purpose Build the topology from a snapshot. Streams particle coordinates (cropped/decimated if requested), rebases cropped coords to the crop origin (so periodic tessellation stays local), runs Delaunay to get DTFE density, runs MSE to identify walls/filaments/clusters, and exports VTK files shifted back to global coordinates.

How it works

1. *Coordinate streaming & decimation* — Read coordinates in chunks (default 2M) to avoid loading all particles at once. Apply crop (`--crop-box`) to

restrict to a sub-volume. Use `--stride 1` to keep all particles in the crop, or `--target-count N` to auto-compute a stride that retains approximately N points.

2. *Rebase crop for periodic tessellation* — Subtract crop minima so coordinates start at (0,0,0); set the NDfield BBOX to the crop size. This keeps `delaunay_3D -periodic` wrapping only within the crop. The crop origin is stored for shifting VTKs back later.
3. *Delaunay tessellation (DTFE density)* — Run `delaunay_3D` on the rebased NDfield (`--delaunay-btype periodic` recommended). Produces a Delaunay tessellation with DTFE density per vertex/cell. MSE then operates on this scalar field. No persistence filtering at this stage.
4. *MSE (topology extraction)* — Run `mse` on the Delaunay NDnet with `--mse-nsig` (higher = prune more), `--dump-manifolds` (wall manifolds, e.g., JE1a), `--dump-filament-manifolds` (optional), `--dump-arcs U` (filament skeleton), and `--dump-clusters` (maxima critical points). When multiple dumps are requested the MSC is reused (`-loadMSC`) to avoid recomputation. Common manifold tag choices:
 - `--dump-manifolds JE1a` → wall manifolds (2D sheets)
 - `--dump-filament-manifolds JE2a` → filament-bounding manifolds
 - `--dump-arcs U` → filament arcs (1D skeleton)
 - `--dump-clusters JE3a` → cluster critical points (maxima)
5. *Conversion & shift back* — Convert manifolds/skeletons with `netconv/skelconv` (optional smoothing via `--netconv-smooth / --skelconv-smooth`). Shift exported VTKs back to global coordinates by adding the stored crop origin. NDnet/NDskl files stay in rebased coordinates.

Key inputs HDF5 snapshot; optional crop box; stride/target-count; `--delaunay-btype`; persistence (`--mse-nsig`); manifold/skeleton tags; smoothing counts; `--export-delaunay` to also export the Delaunay VTU.

Example (cropped, periodic, smoothed exports):

```
python scripts/analyze_snapshot.py \
--input data/snap_010.hdf5 \
--output-dir outputs/snap_010_subbox \
--crop-box 500000 500000 100000 1000000 1000000 200000 \
--stride 1 \
--delaunay-btype periodic \
--export-delaunay \
--mse-nsig 5.0 \
--dump-manifolds JE1a \
--dump-filament-manifolds JE2a \
--dump-clusters JE3a \
```

```
--dump-arcs U \
--netconv-smooth 20 \
--skelconv-smooth 20
```

Outputs (per crop) - Native (rebased coords): <prefix>.NDnet, <prefix>_s*.NDskl. - VTK (shifted to global): <prefix>_delaunay_S###.vtu, <prefix>_manifolds_<TAG>_S###.vtu, <prefix>_arcs_<ARC>_S###.vtp (plus .vtu), optional <prefix>_filament_manifolds_<TAG>_S###.vtu, and <prefix>_cluster_critpoints_<TAG>_S000.vtp/.vtu. - <prefix>_summary_stats.csv (VTK scalar stats).

Why it matters Reduces the dataset to a safe size, builds the DTFE density that MSE needs, applies persistence filtering for meaningful walls/filaments, and delivers globally aligned VTKs for the next stages without periodic tiling artifacts in cropped runs.

Stage 2: `ndtopo_stats.py` (overlap/mass attribution)

Purpose Compute how many Delaunay vertices belong to walls, filaments, clusters, overlaps, or neither, and aggregate scalar sums/means for those categories. Scalars always come from the Delaunay mesh; ID fields and `cell` handling are configurable.

How it works

1. *Optional conversion* — With `--write-vtk`, auto-converts native NDnet/NDskl to unsmoothed VTU/VTP via `netconv/skelconv` if VTK files are not present.
2. *ID matching* — Match structure IDs to Delaunay vertex IDs:
 - Walls Delaunay: `true_index` field (exact match).
 - Filaments Delaunay: integer part of the `cell` field (critical-point index). Many arc samples are interpolated and have no matching Delaunay node.
 - Clusters Delaunay: `true_index` assigned via nearest-neighbor mapping from cluster critical points to Delaunay vertices.
3. *Category aggregation* — Split Delaunay vertices into: `walls`, `filaments`, `walls_not_filaments`, `filaments_not_walls`, `shared_walls_filaments`, `unassigned`, plus filament-manifold variants (`filament_manifolds`, `shared_walls_filament_manifolds`, etc.) and cluster variants (`clusters`, `clusters_not_filaments`, `shared_walls_clusters`, `shared_walls_filaments_clusters`, etc.) when those inputs are provided. Aggregate `--scalar-fields` (default: `mass`, `field_value`, `log_field_value`) over each category.

Key flags - `--write-vtk` — convert NDnet/NDskl to unsmoothed VTU/VTP if needed - `--delaunay-id-field true_index`, `--walls-id-field true_index`, `--filaments-id-field cell` - `--delaunay-cell-mode all|zero`, `--walls-cell-mode all|zero`, `--filaments-cell-mode all|zero` - `--scalar-fields mass field_value log_field_value` - `--per-point-csv PATH` — write one row per Delaunay vertex with boolean membership flags (required for Stage 5 aggregation) - `--netconv-bin/--skelconv-bin` if conversion binaries are not on PATH

Example (per crop):

```
python scripts/ndtopo_stats.py --verbose \
--delaunay-vtk outputs/snap_010_subbox/snap_010_delaunay_S000.vtu \
--walls-vtk      outputs/snap_010_subbox/snap_010_manifolds_JE1a_S020.vtu \
--filaments-vtk outputs/snap_010_subbox/snap_010_arcs_U_S020.vtp \
--cluster-manifolds-vtk outputs/snap_010_subbox/snap_010_cluster_critpoints_JE3a_S000.vtp \
--delaunay-id-field true_index --walls-id-field true_index --filaments-id-field cell \
--delaunay-cell-mode all --walls-cell-mode all --filaments-cell-mode all \
--scalar-fields mass field_value log_field_value \
--output-csv outputs/snap_010_subbox/topology_stats.csv \
--per-point-csv outputs/snap_010_subbox/topology_points.csv
```

Outputs - `<prefix>_topology_stats.csv` — counts and `<scalar>_sum/<scalar>_mean` per category. - `<prefix>_topology_points.csv` (optional, `--per-point-csv`) — one row per Delaunay vertex with boolean membership flags and scalar values. Required input for Stage 5.

Why it matters Shows how much mass/field value sits in overlapping vs unique topology membership, using the authoritative Delaunay values rather than interpolated manifold/skeleton scalars. Filament counts can exceed matched mass because many filament samples are interpolated along arcs and have no Delaunay node — this is expected.

Stage 3: `batch_clip.py` (slab clip + 2D/3D views)

Purpose Take the walls/filaments/density VTKs and make slab-focused products: clipped 3D, flattened 2D, averaged density, and PNGs. Must be run via `pvpython`.

How it works

1. *Clip slab* — Load walls/filaments/density (plus optional filament/cluster manifolds or cluster critical points); clip to a slab (e.g., `--slab-axis z`, `--slab-origin 0`, `--slab-thickness 10` in mpc/h). Uses clipped bounds so resampling never samples outside the crop.
2. *Flatten / average / resample* — Flatten walls/filaments onto the slab

plane (set slab axis coordinate to zero). Average density across the slab thickness. Resample to a regular grid (`--resample-dims`, e.g., $512 \times 512 \times 64$ or $500 \times 500 \times 100$). The scalar to average is set by `--scalar-name` (default `log_field_value`).

3. *Render PNGs (optional)* — 3D slices and 2D flats for walls, filaments, density; composite overlays layer density with walls/filaments. PNG settings include `--png-resolution` (default 1600×1200), `--png-colormap` (default `Inferno` (`matplotlib`)), and `--composite-opacity` (default 0.6).

Key inputs Walls VTU, Filaments VTP, optional filament manifolds VTU, optional cluster manifolds/critical points (VTU/VTP), Delaunay VTU; slab axis/origin/thickness; resample dims; scalar name; `--save-pngs`.

Example:

```
pvpython scripts/batch_clip.py \
--input-dir outputs/snap_010_subbox \
--walls snap_010_manifolds_JE1a_S020.vtu \
--filaments snap_010_arcs_U_S020.vtp \
--delaunay snap_010_delaunay_S000.vtu \
--output-dir outputs/snap_010_subbox/slab_z0 \
--output-prefix snap_010 \
--slab-axis z --slab-origin 0 --slab-thickness 10 \
--resample-dims 512 512 64 \
--scalar-name log_field_value \
--save-pngs \
--png-colormap "Inferno (matplotlib)" \
--png-log-range -3 1 \
--no-png-transparent \
--png-align-composite
```

Notes: - Use `--no-png-transparent` if you want the background color to be visible in PNGs. - `--png-align-composite` (default) aligns composite overlays to density bounds; disable with `--no-png-align-composite` if overlays already share the same coordinate frame.

Outputs (per slab) - 3D clips: `_walls_3d.vtu`, `_filaments_3d.vtp`, `_density_3d.vtu/.vti` - 2D flats/averages: `_walls.vtu`, `_filaments.vtp`, `_walls_filaments.vtm`, `_density.vtu`, `_density_avg.vti` - `_summary_stats.csv` - PNGs if `--save-pngs`

Why it matters Provides ready-to-view 2D/3D slices aligned to a slab, and flattened 2D layers that make overlaps/absence between density, walls, and filaments easy to see at a glance.

Stage 4: `batch_crop_and_clip.py` (batch crops + slabs)

Purpose Tile a large snapshot into non-overlapping crops, run Stages 1–3 on each crop, then run Stage 3 (slab clip) over multiple slab origins inside each crop. Outputs are nested by crop and slab.

How it works

1. *Tile snapshot* — Set `--crop-size` (e.g., $500000 \times 500000 \times 100000$ kpc/h) and ranges (`--x-range`, `--y-range`, `--z-range`). Iterate non-overlapping tiles; skip empty/partial edge tiles.
2. *Per-crop: analyze* — Run `analyze_snapshot.py` with chosen persistence and smoothing (`--mse-nsig`, `--dump-manifolds`, `--dump-filament-manifolds`, `--dump-clusters`, `--dump-arcs`, `--netconv-smooth`, `--skelconv-smooth`).
3. *Per-crop: stats* — Run `ndtopo_stats.py` to write `<crop_prefix>_topology_stats.csv`. If `--write-per-point-csv` is set, also writes `<crop_prefix>_topology_points.csv` for Stage 5 aggregation.
4. *Per-slab: clip* — March slab origins by `--slab-step` / `--slab-thickness`; call `batch_clip.py` with `--resample-dims` and `--scalar-name` to produce 3D/2D clips and PNGs. Use `--skip-slabs` to run analysis + stats only.

Inputs/tiling - `--snapshot`: full HDF5 snapshot. - `--crop-size DX DY DZ`: tile size in `--input-unit` (default kpc/h). - `--x-range/--y-range/--z-range`: overall region to tile; partial edge tiles are skipped. - Units follow `--input-unit` (default kpc/h); converted to `--output-unit` for slab origins (default mpc/h).

Example:

```
python scripts/batch_crop_and_clip.py \
  --snapshot data/snap_010.hdf5 \
  --output-root outputs/quijote_batches \
  --crop-size 500000 500000 100000 \
  --x-range 0 1000000 --y-range 0 1000000 --z-range 0 1000000 \
  --slab-step 10 --slab-thickness 10 \
  --mse-nsig 5.0 \
  --dump-manifolds JE1a --dump-filament-manifolds JE2a \
  --dump-clusters JE3a --dump-arcs U \
  --netconv-smooth 20 --skelconv-smooth 20 \
  --resample-dims 500 500 100 \
  --scalar-name log_field_value \
  --write-per-point-csv
```

Output layout

```
output-root/
  crop_x.../                                # per-crop outputs
```

```

crop_..._topology_stats.csv
crop_..._topology_points.csv      # if --write-per-point-csv
crop_..._delaunay_S###.vtu
crop_..._manifolds_<TAG>_S###.vtu
crop_..._arcs_<ARC>_S###.vtp
crop_..._cluster_critpoints_<TAG>_S000.vtp/.vtu
slab_z.../
    <prefix>_walls_3d.vtu
    <prefix>_filaments_3d.vtp
    <prefix>_density_3d.vtu/.vti, _density.vtu, _density_avg.vti
    <prefix>_walls.vtu, <prefix>_filaments.vtp, <prefix>_walls_filaments.vtm
    <prefix>_summary_stats.csv
PNGs if --save-pngs

```

Why it matters Automates running the full pipeline over many tiles and slabs, organizing outputs cleanly per crop/slab and delivering both visuals and stats for downstream analysis.

Tips - Ensure `pvpython` is on `PATH` for slab clipping; ensure `vtkmodules` is available for VTK shifts/stats. - If a crop contains no particles, it is skipped. - Filament mass < count is expected when many filament `cell` IDs don't match Delaunay `true_index`.

Stage 5: `aggregate_topology_points.py` (cross-crop aggregation)

Purpose Combine `*_topology_points.csv` files from all crops into a single aggregated stats CSV, per-category histograms (CSV + PNG), and composition/violin/box plots for the four primary categories: **Clusters**, **Filaments** (filament manifolds not clusters), **Walls** (walls only), and **Unassigned**.

How it works Scans `--root` recursively for `*_topology_points.csv` files (or uses an explicit `--inputs` list). Three aggregation engines: - **polars** (default) — fast, exact stats. Use `--polars-chunks N` to split into N chunks for very large datasets (approx. quantiles in chunked mode). - **stream** — memory-light, approximate quantiles. - **python** — exact but slow.

Key inputs `--root` pointing to the batch output directory; `--output-dir` and `--output-prefix`; `--engine polars`; `--hist-bin-mode global` for shared bins across categories.

Example:

```

python scripts/aggregate_topology_points.py \
  --root outputs/quijote_batches \
  --output-dir outputs/quijote_batches/combined \
  --output-prefix quijote_batches \

```

```
--engine polars \
--polars-chunks 4 \
--log10-field-value \
--violin-scalar log10_field_value \
--hist-bin-mode global \
--hist-percentile-range 1 99 \
--plot-percentile-range .1 99.9 \
--plot-fontscale 1.2 \
--plot-dpi 300
```

Outputs - `<prefix>_topology_stats.csv` — aggregated stats across all crops.
- `<prefix>_<category>_<scalar>_hist.csv/.png` — per-category histograms (with count and median line).
- `<prefix>_filman_walls_composition.png` — composition barchart (clusters / filaments / walls / unassigned).
- `<prefix>_filman_walls_<scalar>_violin.png` and `_box.png` — distribution plots.

Why it matters Gives a global view of how mass and density are distributed across cosmic-web categories for the full simulated volume, not just individual crops.

Visualization Scripts

These scripts produce 3D animations and comparisons. They are not part of the batch pipeline but consume its outputs.

`spin_render.py` (rotating 3D render)

Orbits a camera around any VTK dataset (VTU/VTP/VTI/VTR) and writes a PNG sequence or MP4. Must be run via `pvpython`. Supports sinusoidal elevation/zoom animation (`--animate-elev-zoom`), multi-loop rotation (`--loops`), and slow modulation across the full movie (`--slow-elev-amp`, `--slow-zoom-amp`).

```
pvpython scripts/spin_render.py \
--input outputs/crop_x0-500_y0-500_z0-100/crop_manifolds_JE1a_S020.vtu \
--output outputs/spin.mp4 \
--frames 360 \
--scalar log_field_value \
--colormap "Inferno (matplotlib)" \
--range-min -3 --range-max 1 \
--resolution 1920 1080 --background black \
--animate-elev-zoom --loops 2
```

Note: default colormap is `Fast`; use `--colormap "Inferno (matplotlib)"` to match the rest of the pipeline.

redshift_evolution.py (redshift evolution movie)

Takes one Delaunay VTU per redshift snapshot and produces a smooth interpolation movie (or slideshow) across redshifts. Must be run via **pvpython**.

Rendering modes:

Mode	Flag	Description
Interpolation	<i>(default)</i>	Smoothly interpolates positions and scalars between snapshots. Requires scipy .
Slideshow	--slideshow	No interpolation — holds each snapshot for --slideshow-duration seconds.

```
pvpython scripts/redshift_evolution.py \  
  --inputs snap_000_delaunay.vtu snap_001_delaunay.vtu \  
           snap_002_delaunay.vtu snap_003_delaunay.vtu snap_004_delaunay.vtu \  
  --labels "z=3" "z=2" "z=1" "z=0.5" "z=0" \  
  --output-dir outputs/evolution --output-prefix evolution \  
  --scalar log_field_value \  
  --range-min -3.0 --range-max 1.0 \  
  --slideshow --slideshow-duration 2.0
```

Notes: - Proximity alignment (chained KDTree nearest-neighbor) ensures interpolation traces the same physical particle across snapshots rather than position-index correspondence. Requires **scipy**; falls back to positional ordering with a warning if unavailable. - Label position is in normalized viewport coordinates (**--label-position X Y**); default 0.45 0.05 is center-bottom.

compare_simulations.py (side-by-side comparison)

Reads two simulations' **topology_stats.csv** files and renders side-by-side boxplots across the four primary categories. The color scheme matches **aggregate_topology_points.py** (clusters yellow, filaments orange, walls red, unassigned purple).

```
python scripts/compare_simulations.py \  
  --folder-a outputs/quijote_batches_000_w_clusters_points \  
  --folder-b outputs/quijote_batches_004_w_clusters_points \  
  --labels "z=3" "z=0" \  
  --scalars log_field_value \  
  --output-dir outputs/comparison --output-prefix compare_z3_vs_z0
```

Data Analysis Outputs

- **VTK files:** shifted walls/filaments/density (3D and flattened 2D), plus averaged density grids for slice/overlay inspection in ParaView.
 - **PNGs:** 3D slices, 2D flats, and composites that show layered density/walls/filaments to visually assess overlap and separation.
 - **Summary stats:** Per-file scalar stats (`*_summary_stats.csv`) and per-crop topology overlap stats (`*_topology_stats.csv`) to quantify counts, sums, and means in shared/unique/unassigned categories.
 - **Per-point stats:** `*_topology_points.csv` files (one row per Delaunay vertex) support histogramming and cross-crop aggregation in Stage 5.
 - **Cross-crop aggregation:** Global stats CSV, per-category histogram CSVs/PNGs, composition barchart, violin/box plots produced by `aggregate_topology_points.py`.
 - **Simulation comparison:** Side-by-side boxplots from `compare_simulations.py` for direct comparison across runs/redshifts.
 - **3D animations:** Rotating renders (`spin_render.py`) and redshift evolution movies (`redshift_evolution.py`).
-

Troubleshooting / Gotchas

- **VTK import errors:** Ensure `vtkmodules` is on `PYTHONPATH` (via conda environment hooks) and ParaView's Python is discoverable when using `pvpython`.
 - **HDF5 Blosc filter:** Install `hdf5plugin` in the environment that runs `analyze_snapshot.py`; otherwise Quijote HDF5 loads fail.
 - **Interpolated filament IDs:** `filaments_not_walls` counts can exceed mass because many filament arc samples are interpolated and have no Delaunay node. This is expected — filament mass < filament count is normal.
 - **Clipping artifacts:** Use slab bounds for resampling (`batch_clip.py` already does). Verify crop rebasing/shift if density extends beyond the tile.
 - **File naming:** Stems no longer duplicate format tokens; if older files remain, prefer the non-duplicated names for downstream steps.
 - **pvpython PATH:** Ensure `pvpython` is on `PATH` before running `batch_clip.py`, `spin_render.py`, or `redshift_evolution.py`. See the README for the conda activate.d hook approach.
 - **--periodic vs --delaunay-btype periodic:** Use `--delaunay-btype periodic` for cropped runs. `--periodic` can introduce tessellation artifacts.
-

Technologies Used

Category	Tool
Hardware	Apple M1 Pro, 16 GB, macOS Tahoe
Data transfer	Globus (CLI)
Topology extraction	DisPerSE (<code>delaunay_3D</code> , <code>mse</code> , <code>netconv</code> , <code>skelconv</code>)
Visualization	ParaView / <code>pvpython</code>
Scripting	Python, <code>pvpython</code>
IDE	VS Code, Positron
Documentation	Quarto (Markdown), Mermaid (flowcharts)
Version control	Git, GitHub
AI assistance	ChatGPT, Codex, Claude Code, GitHub Copilot

Materials / Sources

- Quijote snapshot (local: `data/snap_010.hdf5` or `data/snapdir_000/`).
- DisPerSE: <https://www.iap.fr/useriap/sousbie/web/html/indexd41d.html>
- ParaView / `pvpython`: <https://www.paraview.org/>
- Python scripts: in `scripts/` (this repository).
- Dependencies: `hdf5plugin`, `vtkmodules`, `numpy`, `h5py`, `polars`, `scipy`.